

Version control with git and GitHub

Week 05 workshop

BAN405 Python Programming for Data Science

Why

A familiar problem

```
convert.py  
convert_v2.py  
convert_v2_fixed.py  
convert_final.py  
convert_final_v3_ACTUAL.py
```

- It worked yesterday. Today it does not.
- Which file is the one that worked?
- What changed between them, and why?

Every programmer has a folder like this.
Version control is the fix.

Why not just OneDrive?

A **sync service** keeps

- the *latest* state of every file
- older versions by *time*, saved whenever it felt like it
- nothing about *what* changed or *why*

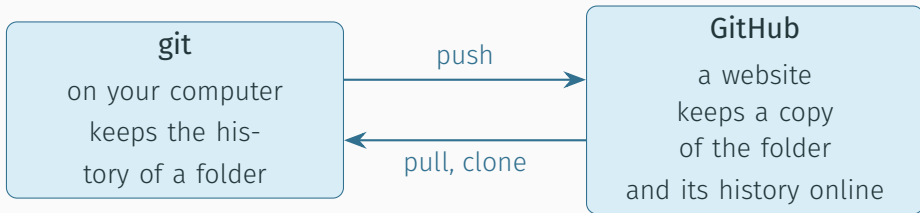
git keeps

- snapshots *you* decide to take
- each with a message saying *why*
- and the exact lines that changed

A backup answers “*can I get my file back?*”

Version control also answers “*what changed, when, and why?*”

git and GitHub are two different things



- A **repository** is a folder whose history git keeps.
- A **commit** is one snapshot in that history.

What an analyst gets out of it

1. **Explain what changed.** *“The report said 12.4 % last week, and 11.9 % now. Why?”*
2. **A backup**, and the same project on more than one computer.
3. **Something to share**: a link instead of an email attachment, and a portfolio.
4. **Working together** on one project without emailing files back and forth.

git is the standard tool for all four, in software and in data teams alike.

Five exercises. Each adds *one* new thing you can do.

	New skill	Where
1	Read a history	on github.com
2	Clone and commit	a copy of someone else's project
3	Start a repository	your own BAN405 folder
4	Push	your BAN405 folder → GitHub
5	Pull	your repository on two computers

Everything happens in Positron's **Source Control** panel.

Explanations, options and fixes are in the [git guide](#), in the **guides** folder.

Reading a history

Exercise 1 — Read a history

EXERCISE

- A repository on GitHub holds the temperature converter you wrote yourself, in six commits.
- Look through the commits and their diffs.
- Then find the commit that made 212 °F come out as 85.8 °C.

Steps

All steps and checks are in [exercises.md](#). You only need a browser.

What you just saw

- A **commit**: a message, an author, a date, and a **hash** such as 4803c4f.
- A **diff**: the lines that changed. **Red** removed, **green** added.

The lesson

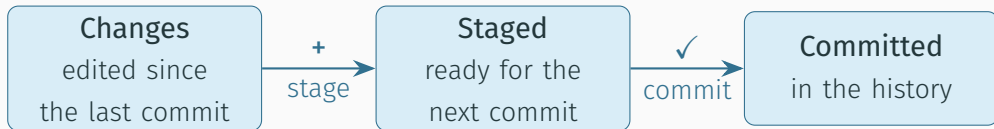
The commit was called *“Tidy up formatting”*. It also changed the formula.

The **message** is what someone meant to do. The **diff** is what they did.

Without the history you would be re-reading the whole program. With it, the answer is one click away.

Making commits

The three places a change can be



- **Staging** lets you choose what goes into a commit, instead of everything at once.
- **Discard Changes** throws away everything since your last commit. It is the undo you will use most.

Not everything belongs in the history

A **.gitignore** is a text file listing what git should never track.

Keep out

- files your computer makes by itself (`__pycache__`, `.DS_Store`)
- files your code produces (figures, tables)
- **secrets**: passwords and keys
- **personal data** about other people

The rule

Every new repository starts with the course template:

`guides/gitignore-template.txt`

Copy it into `.gitignore`, and commit it first.

Once something is pushed, assume it is public, even in a private repository. Deleting it later leaves it in the history.

Exercise 2 — Clone it and fix it

EXERCISE

- **Together:** tell git who you are (the *one* terminal step today), and clone the temperature converter into a `git-practice` folder.
- **On your own:** keep a secret out of git with a `.gitignore`, fix the bug, and commit both.
- Break something on purpose, and undo it.

Steps

All steps and checks are in [exercises.md](#). Mac: **Terminal**. Windows: **Git Bash**.

- Commit when you finish something, not when you are happy with everything.
- One commit, one change. A fix and a new feature are two commits.
- Write the message for whoever reads the history next. Usually you.

✗ fix stuff	✓ Fix crash when the input is empty
✗ update	✓ Add absolute-zero check to the converter
✗ final	✓ Split the analysis into functions

Your own repository

Where the history lives: `.git`

```
git-practice/  
└─ temperature-converter/  
   │├─ .git/   ← the history  
   │├─ .env     ← ignored  
   │├─ .gitignore  
   │├─ convert.py  
   │├─ README.md  
   └─ temperature_tools.py
```

- A **repository** is a folder plus its hidden `.git` folder: the whole history.
- Names starting with a dot are hidden by default.
- Delete `.git`, and git forgets the history. Your files stay.

Together: show hidden files

Windows: File Explorer → **View** → **Show** → **Hidden items**

Mac: in Finder, press **Cmd+Shift+.**

One repository: your whole BAN405 folder

your computer



1. **Initialize once**, in the BAN405 folder itself.
2. **Never** start a repository inside one of its folders.
3. **Always open the BAN405 folder** in Positron, not a week folder.
4. **Clone other projects next to it**, never inside it.

Exercise 3 — Start your own repository

EXERCISE

- Open your BAN405 folder in Positron, and initialize a repository.
- Add the `.gitignore` template.
- Make your first commit.

Steps

All steps and checks are in [exercises.md](#).

GitHub

Push: give your history a copy online

- **Push** sends your commits to GitHub. Until then, they exist only on your computer.
- **Private** repositories are visible only to you. **Public** ones, to anyone.
- Positron signs you in to GitHub through your browser: no passwords in a terminal.

Positron's words for it

Publish Branch	create the repository on GitHub, then push
Sync Changes	pull, then push

Exercise 4 — Put it on GitHub

EXERCISE

- Publish your BAN405 repository as a **private** repository.
- Add a README, commit it, and push.
- Look at the result on github.com.

Steps

All steps and checks are in [exercises.md](#).

Two copies of one repository



Copies do not update each other. Two habits keep them in step:

- **Pull** when you start working.
- **Push** when you finish, even if the work is unfinished.

Exercise 5 — Work from a second computer

EXERCISE

- You have one computer here, so **github.com stands in for the second**: an edit there is a commit your computer has not seen, just like a push from your other laptop.
- Edit the README on github.com, then **pull** it into Positron.
- *Optional*: forget to pull, and see what happens.

Steps

All steps and checks are in [exercises.md](#). Editing on github.com is only a stand-in: in your own work, change files on your computer and push.

When copies disagree

Merge conflicts

A conflict needs **two copies that both changed the same lines: you on two computers**, or **two people** in one repository. git will not guess which version is right, so it marks the spot and asks you.

Avoiding them

1. Pull when you start, push when you finish.
2. Commit small and often.
3. Divide the work by file.
4. One person per notebook. Always.

Notebooks store their output as well as their code. Never merge one by hand: keep *one* whole version, and redo the other change.

Demo: forgetting to pull

LIVE DEMO

1. **On the other computer** (github.com stands in): change a line of `README.md`, and commit.
2. **On this computer, without pulling**: change the *same* line differently. Stage, commit.
3. **Sync Changes**. git pulls first — and stops. `README.md` is listed under **Merge Changes**.

Why

Both copies changed the same line since they were last in step. git will not guess which one is right, so it asks you.

Demo: resolving the conflict

LIVE DEMO

git marks the spot in the file:

```
<<<<<< HEAD
```

```
My work, edited on this computer.
```

```
=====
```

```
My work, edited on the other one.
```

```
>>>>>> 7bdf541...
```

Current is yours. *Incoming* is what you pulled.

4. Choose **Accept Current**, **Accept Incoming** or **Accept Both**, or edit by hand. No marker lines may remain.
5. Save, stage with **+**, and commit. The message is written for you.
6. **Sync Changes**. In Positron's **Graph**, the two lines of history join in a **merge commit**.

From now on

Commit as you work. Push when you finish.

- Your BAN405 repository is where your work for the rest of the course lives.
- Pull when you start, if you use more than one computer.
- When something goes wrong: the [git guide](#), section *Troubleshooting*.

What we did not cover

Today was the basics. Worth knowing these exist, and the git guide covers each:

- **Fork**: a copy of someone else's repository in *your* GitHub account. A clone lives on your computer; a fork lives on GitHub.
- **Pull request**: asking a repository's owner to accept your changes. How open-source software is built.
- **Branches**: parallel lines of history, so a team can work on several things at once.
- **Template**: a new repository starting from someone else's files, with no history.
- **git in the terminal**: everything you did today, typed. Most tutorials use it.

What you did today

Today you

- found a bug by reading a history
- cloned a project and committed a fix
- kept a secret out with `.gitignore`
- put your BAN405 folder on GitHub
- kept two computers in step

At home (in [exercises.md](#))

- Commit your earlier work, one exercise per commit.
- *Optional*: cause a merge conflict with a classmate.

Next week

The terminal, paths and conda: where your files really are, and how to make a project run on someone else's computer.